Asynchronous JavaScript

Lec-13

Made with GAMMA

How JS Works Behind the Scene

- **JavaScript Execution Context** : It is an abstract environment created by JavaScript Engine, to execute and evaluate JavaScript code.
- In JavaScript Execution context are of two types:

1. **Global Execution Context (GEC)** : Creates automatically by the JS engine when your script runs first. **Except the code inside any function** GEC **handles all basic level code** and **creates global object (eg. window object in browsers)**. There is only one GEC per script execution.
2. **Function Execution Context (FEC)** : Get's created dynamically everytime a function is invoked. Each function call get's its **own independent context** to execute it's local code.

- Whenever JavaScript Execution Context is generated, it runs in two different phases:

1. **Memory Creation Phase**
2. **Execution Phase**

Let's try and breakdown how below given code will execute:

Step-1 : **Global Execution Context (GEC)** get's created and get's allocated to 'this'.

Step-2 : **Memory Creation Phase** get's created, in which all the variable get's collected and stored in it.

val1 ← undefined

val2 ← undefined

addVal← fn definition

res1 ← undefined

res2 ← undefined

This concludes our Memory creation phase

Step-3 : **Execution phase** starts

val1 ← 10

val1 ← 20

addVal ← it will **create it's own execution context**, in which **new variable environment** and **Execution thread** get's generated



since new sandbox is now created for **addVal()**, **it would again create Memory Creation Phase and Execution Phase** for this new sandbox.

Step- 3.1 : **Memory creation phase** of addVal()

num ← undefined

num 1← undefined

total ← undefined

Step- 3.2 : Execution phase of addVal()

num ← 10

num1 ← 20

total ← 30

After this new execution Context created for addVal() get's destroyed.

Step-3 Continues:

res1 ← 30

Made with GAMMA

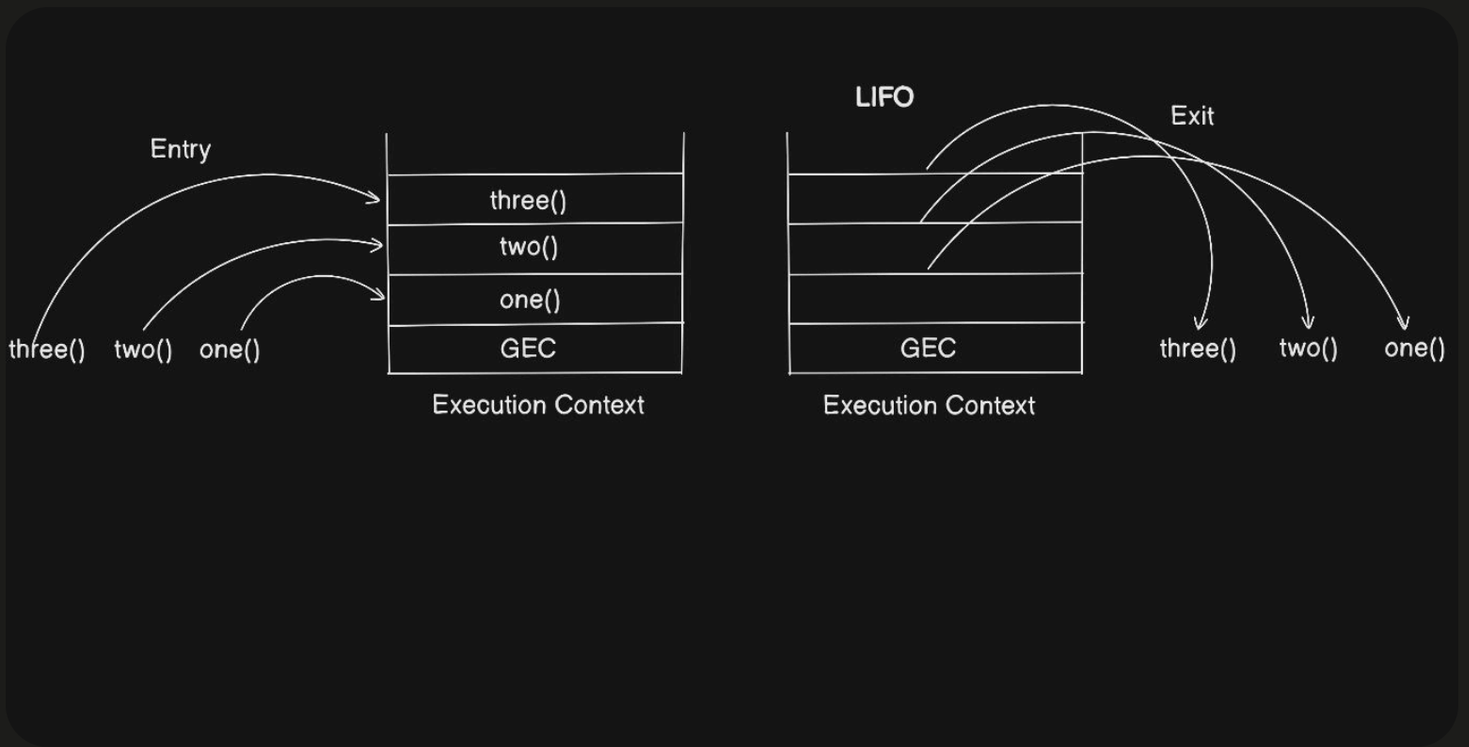
Call Stack

- The call stack is a data structure that JavaScript uses to keep track of function call and execution contexts in a program.
- It operates on a **LIFO (Last-In, First-Out)** basis, so the function called last finishes first.
- Since, JavaScript is **single-threaded**, meaning it **has only one call stack and can execute only one thing at a time**.
- The stack remembers which function is running and where to return after it finishes.

Eg:

```
// Call Stack
function one(){
    console.log("one");
    two()
}
function two(){
    console.log("two");
    three()
}
function three(){
    console.log("three");
}

one()
two()
three()
```



Made with GAMMA

What is Asynchronous JavaScript ?

- Generally, **JavaScript runs one task at a time**.
- While **one task is running**, **no other JavaScript code can run**.
- If a task takes a long time, the browser cannot respond to user actions until the task is finished.
- Asynchronous programming solves this problem **by allowing long-running operations to complete in the background while JavaScript continues running other code**.
- The word **"async"** means - **"not at same time"**.
- Instead of waiting for one task to finish before starting the next, JavaScript can continue running other code while waiting for an operation to complete.
- When the operation finishes, JavaScript continues by running code that handles the result.

Why Asynchronous Programming ?

- Some operations take time to complete like- **downloading files, waiting for timers, waiting for events** or **waiting for network requests**, etc.

Timers	Runs after specified milliseconds
Events	Runs when triggered by an event
Network requests	Runs when data arrives.

- If JavaScript waited for these tasks, the page would freeze, **a frozen page is an broken page**.
- To stay **responsive**, JavaScript uses **async programming.**
- **Asynchronous flow** refers to **how JavaScript allows certain operations to run in the background and let their results be handled when they are ready.**

Made with GAMMA

Asynchronous Timer Functions

- JavaScript timers let you run a function **after a delay** or repeatedly **at fixed intervals**.
- **Timer functions do not pause the running JavaScript**.
- JavaScript immediately continous with the next statement.
- The two main timer functions are **setTimeout()** and **setInterval()**

1. setTimeout()

- The **setTimeout()** function **schedules a function to run once after a delay**.
- It is an **Higher Order Function**, as it **takes an callback function as one of it's arguments**.
- **Syntax**:

```
setTimeout(function, milliseconds)
```

- In above syntax, **first argument** is the **callback function** to run.
- The **second argument** is the **delay in milliseconds**.
- **1 sec = 1000 milliseconds**
- **Eg:**

```
console.log("Wait for a message after 2 sec");

 // setTimeout()
 setTimeout(()=>{
     console.log("Hello I am Asynchronous message");
  },2000)
```

- If we add **0 as second argument** for a setTimeout() function, in such case the **callback will execute after the JavaScript has been completely executed**.

Made with GAMMA

Asynchronous Timer Functions

2. clearTimeout()

- The **setTimeout() function returns a timer identifier**.
- We can **pass the timer identifier to clearTimeout() to cancel the setTimeout()**.
- **Syntax**:

```
clearTimeout(timerId)
```

- **Eg:**

```
const startBtn = document.getElementById('start-btn')
const stopBtn = document.getElementById('stop-btn')
let timerDisplay = document.getElementById('timer-display')
let timerBehaviour;

startBtn.addEventListener('click', ()=>{
            let timerBehaviour = setTimeout(()=>{
            timerDisplay.innerText = 'Timer Started after 1 sec'
          },1000)
    })

 stopBtn.addEventListener('click',()=>{
             clearTimeout(timerBehaviour)
             timerDisplay.innerText = 'Timer Stopped'
 })
```



On clcking **'Start Timer'**



On clicking **'Stop Timer'**



Practice Question

You're building an idle-session handler for a web app. **If the user doesn't interact with the page (no mouse move, click, or keypress)** for 10 seconds, show a warning message. If they still don't interact for another 5 seconds after that, log them out automatically. But if the user does anything at all, the whole countdown should reset from zero — the warning should never fire if they're actively using the app.

Write a function **startIdleTimer()** that implements this behavior. Specific requirements:

- **After 10 seconds of inactivity**, log **"Warning: You will be logged out due to inactivity"** to the console.
- **After 15 seconds total of inactivity (5 seconds past the warning)**, log **"User logged out"**.
- Listen for **mousemove, keydown, and click events on the document** — any of these should reset both timers back to zero.
- If the **user keeps interacting continuously, neither message should ever print**.
- Make sure **you're not stacking up timers with every event — each reset should clean up the previous ones properly**.

Made with GAMMA

Asynchronous Timer Functions

3. setInterval()

- The **setInterval()** function **runs a callback function repeatedly**.
- It is also an **Higher Order Functions**.
- The callback function here is called repeately with (at least) specified delay in between.
- **Syntax**:

```
setInterval(function, milliseconds)
```

- **Eg**:

```
setInterval(() => {
  let h3 = document.createElement("h3");
  let thirdHeadingText = document.createTextNode("Hello");
  h3.appendChild(thirdHeadingText);
  document.body.appendChild(h3);
}, 2000);
```

- If we add **0 as second argument** for a setInterval() function, in such case the **callback will execute after the JavaScript has been completely executed.**

Made with GAMMA

Asynchronous Timer Functions

4. clearInterval()

- The **setInterval() function returns a timer identifier**.
- We can **pass the timer identifier to clearInterval() to cancel the setInterval().**
- **Syntax**:

```
clearInterval(intervalId);
```

- **Eg**:

```
let countdownTillZero = setInterval(()=>{
document.body.innerText = `Timer reading
${timeinSec}`
timeinSec--

if(timeinSec < 0){
    clearInterval(countdownTillZero)
    document.body.innerText = 'Timer killed'
}},1000)
```

Timer reading 5

Timer reading 4

Timer killed

Practice Question

You're building an idle-session handler for a web app. **If the user doesn't interact with the page (no mouse move, click, or keypress)** for 10 seconds, show a warning message. If they still don't interact for another 5 seconds after that, log them out automatically. But if the user does anything at all, the whole countdown should reset from zero — the warning should never fire if they're actively using the app.

Write a function **startIdleTimer()** that implements this behavior. Specific requirements:

- **After 10 seconds of inactivity**, log **"Warning: You will be logged out due to inactivity"** to the console.
- **After 15 seconds total of inactivity (5 seconds past the warning)**, log **"User logged out"**.
- Listen for **mousemove, keydown, and click events on the document** — any of these should reset both timers back to zero.
- If the **user keeps interacting continuously, neither message should ever print**.
- Make sure **you're not stacking up timers with every event — each reset should clean up the previous ones properly**.

Made with GAMMA